ULTIMATE FIX COMMAND — TryNex Lifestyle Complete Overhaul

You are a senior fullstack developer. You MUST fix EVERY issue listed below in ONE continuous session. Do NOT stop. Do NOT say "I've improved things." Do NOT partially fix. Complete EVERYTHING from start to finish. Read this ENTIRE prompt before starting, then execute in order.

PROBLEM #1: DESIGN STUDIO — DRAGGING IS COMPLETELY BROKEN

What's wrong: In `DesignStudio.tsx`, the `motion.svg` element has BOTH the `@use-gesture/react` spread handlers AND an explicit `onPointerDown={handleSvgPointerDown}` prop. Because React evaluates JSX props left-to-right, the explicit `onPointerDown` COMPLETELY OVERRIDES the gesture library's internal pointer-down handler. This means `useGesture`'s drag state machine never initializes — users can click to select layers but CANNOT drag/move them.

THE FIX:

1. Find the `motion.svg` element (around line 2949-2960 in DesignStudio.tsx)
2. The spread from `bindCanvasGestures()` must come AFTER any explicit handlers, OR you must create a MERGED handler that calls BOTH:

JSX

```
// WRONG (current code — gesture library is overridden):
<motion.svg
  {...bindCanvasGestures()}
  onPointerDown={handleSvgPointerDown}  // This kills the gesture library
  onWheel={handleCanvasWheel}
  style={{ touchAction: "none" }}
>

// CORRECT FIX — merge both handlers:
const gestureBindings = bindCanvasGestures();

const mergedPointerDown = (e) => {
  handleSvgPointerDown(e);  // Your selection logic runs first
  if (gestureBindings.onPointerDown) {
    gestureBindings.onPointerDown(e);  // Gesture library initializes drag
  }
};
```

```
<motion.svg
  {...gestureBindings}
  onPointerDown={mergedPointerDown}  // Both handlers run
  onWheel={handleCanvasWheel}
  style={{ touchAction: "none" }}
>
```

1. After fixing this, verify that:
   - Clicking a layer selects it (shows handles)
   - Holding and dragging a selected layer MOVES it smoothly
   - Dragging a corner handle RESIZES it (maintaining aspect ratio)
   - Dragging the rotation handle ROTATES it
   - ALL of the above work on BOTH mouse AND touch devices
   - `setPointerCapture(e.pointerId)` is called on pointerdown for smooth tracking

PROBLEM #2: PRINT AREA COORDINATES ARE WRONG

What's wrong: The print zones in `mockups.tsx` (lines 103-122) use a 1000×1000 coordinate system but the values don't match real product dimensions:

- T-Shirt: `{ x: 305, y: 220, w: 390, h: 395 }` — This gives almost 1:1 ratio which is WRONG. Real T-shirt front print is 12"×16" (3:4 ratio)
- Hoodie: `{ x: 332, y: 252, w: 336, h: 278 }` — Too small and wrong ratio
- Cap: `{ x: 338, y: 290, w: 324, h: 240 }` — Ratio should be 4:2.5 (1.6:1)
- Mug: `{ x: 192, y: 248, w: 410, h: 472 }` — Ratio is inverted, should be wider than tall

THE FIX — Update `mockups.tsx` lines 103-122 with CORRECT coordinates:

TypeScript

```
// CORRECT print areas based on real product dimensions (1000×1000 coordinate s

// T-Shirt Front: Real print = 12" × 16" (ratio 3:4, width < height)
// On a 1000×1000 canvas, width should be ~380, height should be ~507 (maintain
TSHIRT_FRONT: { x: 310, y: 180, w: 380, h: 507 }

// T-Shirt Back: Same as front
TSHIRT_BACK: { x: 310, y: 180, w: 380, h: 507 }

// T-Shirt Sleeves: Real print = 3.5" × 4"
TSHIRT_LEFT_SLEEVE: { x: 100, y: 280, w: 140, h: 160 }
TSHIRT_RIGHT_SLEEVE: { x: 760, y: 280, w: 140, h: 160 }
```

```
// Hoodie Front: Real print = 12" × 12" (1:1 ratio, shorter due to pocket)
HOODIE_FRONT: { x: 320, y: 220, w: 360, h: 360 }

// Hoodie Back: Real print = 12" × 14"
HOODIE_BACK: { x: 320, y: 180, w: 360, h: 420 }

// Cap Front Panel: Real print = 4" × 2.5" (wider than tall, ratio 1.6:1)
CAP_FRONT: { x: 310, y: 310, w: 380, h: 238 }

// Mug Wrap: Real print = 8.5" × 3.5" (MUCH wider than tall, ratio 2.43:1)
MUG_WRAP: { x: 150, y: 320, w: 700, h: 288 }

// Water Bottle: Real print = 3" × 8" (taller than wide, ratio 1:2.67)
WATERBOTTLE: { x: 370, y: 200, w: 260, h: 600 }
```

IMPORTANT: After updating these values, visually verify each product by loading the design studio and checking that the dashed border print zone MATCHES the actual printable area visible on the mockup image. Adjust x/y positioning if the zone doesn't align with the product's actual printable surface.

PROBLEM #3: COLOR CHANGE CAUSES SIZE JUMPING

What's wrong: In `mockups.tsx` (around line 326, the `GarmentSVG` component), changing color triggers logic that swaps between a "full studio photo" and a "cutout mask" (lines 350-380). The `isNearBlack` check (line 317) forces a swap to dedicated dark assets. If dark assets have different dimensions/padding than white cutouts, the mockup container JUMPS in size.

THE FIX:

1. Ensure ALL mockup images (every color variant) have EXACTLY the same pixel dimensions and the product is positioned identically within each image
2. If you cannot control image dimensions, force the container to a fixed aspect ratio:

CSS

```
/* In the design studio mockup container */
.mockup-canvas-container {
  position: relative;
  width: 100%;
  max-width: 600px;
  aspect-ratio: 1 / 1;  /* FORCE fixed ratio */
  margin: 0 auto;
}
```

```
.mockup-canvas-container svg {
  width: 100%;
  height: 100%;
}

.mockup-canvas-container img {
  position: absolute;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  object-fit: contain;
}
```

1. In the color-change handler, ONLY swap the image source. Do NOT re-render the container. Do NOT recalculate dimensions. The print area overlay stays in the EXACT same position regardless of color.

PROBLEM #4: UPLOADED DESIGN DOESN'T AUTO-FIT IN PRINT AREA

What's wrong: When a user uploads an image, it doesn't scale to fit nicely inside the print zone. It either overflows, is too small, or loses aspect ratio.

THE FIX — Add auto-fit logic after upload:

TypeScript

```
function autoFitDesignToZone(
  imageNaturalWidth: number,
  imageNaturalHeight: number,
  printZone: { x: number, y: number, w: number, h: number }
) {
  const imageAspect = imageNaturalWidth / imageNaturalHeight;
  const zoneAspect = printZone.w / printZone.h;

  let fitW: number, fitH: number;

  if (imageAspect > zoneAspect) {
    // Image is wider — constrain by width
    fitW = printZone.w * 0.85;  // 85% of zone width (breathing room)
    fitH = fitW / imageAspect;
  } else {
    // Image is taller — constrain by height
    fitH = printZone.h * 0.85;
```

```
    fitW = fitH * imageAspect;
  }

  // Center within the print zone
  const x = printZone.x + (printZone.w - fitW) / 2;
  const y = printZone.y + (printZone.h - fitH) / 2;

  return { x, y, width: fitW, height: fitH, rotation: 0 };
}

// Call this immediately after image upload:
// const fitResult = autoFitDesignToZone(img.naturalWidth, img.naturalHeight, c
// setLayerTransform(newLayerId, fitResult);
```

PROBLEM #5: DESIGN LOOKS LIKE IT'S FLOATING ON TOP (NOT PRINTED ON PRODUCT)

What's wrong: The uploaded design appears as a flat rectangle sitting on top of the product mockup instead of looking like it's actually printed on the fabric/surface.

THE FIX:

1. Apply CSS blend mode to the design layer:

CSS

```
/* For light-colored products (white, cream, light gray) */
.design-layer-light {
  mix-blend-mode: multiply;
  opacity: 0.93;
}

/* For dark-colored products (black, navy, dark gray) */
.design-layer-dark {
  mix-blend-mode: screen;
  opacity: 0.88;
}
```

1. If using SVG (which you are), apply the blend mode via SVG attribute:

XML

```
<image
  href={uploadedDesignUrl}
  style="mix-blend-mode: multiply; opacity: 0.93;"
```

```
  clip-path="url(#printAreaClip)"
/>
```

1. IMPORTANT: The design MUST be clipped to the print area boundary. It should NEVER overflow outside the print zone borders. Use SVG `clipPath` or CSS `clip-path` to enforce this.

PROBLEM #6: SELECTION HANDLES LOOK WRONG (CROP/EXTEND POINTERS)

What's wrong: The user sees ugly crop/extend cursor icons and resize pointers that look confusing. They want Canva-style clean selection handles — simple circles at corners, clean border, no confusing cursors.

THE FIX — Restyle selection handles in DesignStudio.tsx (around lines 3082-3171):

JSX

```jsx
// Replace the current handle rendering with clean Canva-style handles:

{selectedLayer && (
  <g>
    {/* Selection border — thin, blue, slightly rounded */}
    <rect
      x={layerBounds.x}
      y={layerBounds.y}
      width={layerBounds.width}
      height={layerBounds.height}
      fill="none"
      stroke="#0066FF"
      strokeWidth={2}
      strokeDasharray="none"
      rx={2}
      pointerEvents="none"
    />

    {/* Corner resize handles — small white circles with blue border */}
    {['nw', 'ne', 'sw', 'se'].map(corner => (
      <circle
        key={corner}
        cx={getCornerX(corner, layerBounds)}
        cy={getCornerY(corner, layerBounds)}
        r={6}
        fill="white"
        stroke="#0066FF"
```

```
          strokeWidth={2}
          style={{ cursor: 'pointer' }}  // Simple pointer, NOT nwse-resize
          onPointerDown={(e) => handleResizeDown(e, corner)}
        />
      ))}

      {/* Rotation handle — circle above the top center */}
      <circle
        cx={layerBounds.x + layerBounds.width / 2}
        cy={layerBounds.y - 30}
        r={6}
        fill="white"
        stroke="#0066FF"
        strokeWidth={2}
        style={{ cursor: 'grab' }}  // NOT crosshair
        onPointerDown={handleRotateDown}
      />

      {/* Line connecting rotation handle to selection box */}
      <line
        x1={layerBounds.x + layerBounds.width / 2}
        y1={layerBounds.y}
        x2={layerBounds.x + layerBounds.width / 2}
        y2={layerBounds.y - 24}
        stroke="#0066FF"
        strokeWidth={1.5}
        pointerEvents="none"
      />
    </g>
  )}
```

Remove ALL `cursor: "nwse-resize"`, `cursor: "crosshair"`, `cursor: "nesw-resize"` from handles. Use only `cursor: "pointer"` for resize handles and `cursor: "grab"` for rotation.

PROBLEM #7: CLICKING OUTSIDE DOESN'T DESELECT

What's wrong: When user clicks/taps outside the design or print area, the selection should clear but it doesn't.

THE FIX — In `handleSvgPointerDown` (around line 2955):

TypeScript

```
const handleSvgPointerDown = (e: React.PointerEvent) => {
  // Check if the click target is the SVG background (not a layer or handle)
  const target = e.target as SVGElement;
```

```
  // If clicked on the SVG canvas itself (background), deselect everything
  if (target === svgRef.current || target.classList.contains('canvas-background
    setSelectedLayer(null);
    return;
  }

  // Otherwise, proceed with normal selection/drag logic
  // ... existing logic
};
```

Also ensure the SVG has a clickable background rect:

XML

```
<rect
  className="canvas-background"
  x="0" y="0" width="1000" height="1000"
  fill="transparent"
  pointerEvents="all"
/>
```

PROBLEM #8: FULL WEBSITE RESPONSIVENESS FIX

What's wrong: The ENTIRE website has scroll issues, overflow problems, and broken layouts on mobile/tablet/desktop.

THE FIX — Global CSS changes (add to your global stylesheet):

CSS

```
/* === GLOBAL RESET FOR RESPONSIVENESS === */
*, *::before, *::after {
  box-sizing: border-box;
}

html, body {
  overflow-x: hidden;
  width: 100%;
  margin: 0;
  padding: 0;
}

body {
  -webkit-overflow-scrolling: touch;
  overflow-y: auto;
```

```
}

img, video, svg {
  max-width: 100%;
  height: auto;
}

/* === REMOVE THESE PATTERNS EVERYWHERE IN YOUR CODE === */
/* Search and destroy any of these on page-level containers: */
/* overflow: hidden (on main wrappers — this kills scrolling) */
/* height: 100vh (on content wrappers — this prevents scroll) */
/* position: fixed (on non-modal elements) */

/* === DESIGN STUDIO SPECIFIC === */
/* The design canvas area should block scroll (so user can draw) */
/* But the REST of the page should scroll normally */
.design-studio-canvas {
  touch-action: none;  /* Prevent scroll inside canvas */
  overflow: hidden;
}

.design-studio-page {
  overflow-y: auto;  /* Allow page scroll */
  overflow-x: hidden;
  -webkit-overflow-scrolling: touch;
}

/* === RESPONSIVE GRID FOR PRODUCT PAGES === */
.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(260px, 1fr));
  gap: 20px;
  padding: 16px;
}

/* === MOBILE FIXES === */
@media (max-width: 768px) {
  .design-studio-canvas {
    width: 100vw;
    max-width: 100%;
    aspect-ratio: 1;
  }

  /* Stack sidebar controls below canvas on mobile */
  .design-studio-layout {
    flex-direction: column;
  }
```

```
  /* Full-width buttons on mobile */
  button {
    min-height: 44px;  /* Touch target size */
    min-width: 44px;
  }
}
```

Search your ENTIRE codebase for these and FIX:

1. Any `overflow: hidden` on elements bigger than 400px — REMOVE IT (except on the design canvas)
2. Any `height: 100vh` on scrollable content — change to `min-height: 100vh`
3. Any fixed pixel widths on containers — change to `max-width` with percentage fallback
4. Any images without `max-width: 100%` — ADD IT
5. Test scrolling with mouse wheel on EVERY page after fixing

---

PROBLEM #9: ADMIN PANEL — DATABASE "NOT CONNECTED" ERROR

What's wrong: The admin dashboard shows "Database: Not Connected" in some places but connected in others. This is a serverless connection pooling issue.

THE FIX:

1. Find your database client file (likely `lib/prisma.ts` or `lib/db.ts`) and ensure singleton pattern:

TypeScript

```
// lib/prisma.ts (or wherever your DB client lives)
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const prisma = globalForPrisma.prisma || new PrismaClient({
  log: process.env.NODE_ENV === 'development' ? ['error', 'warn'] : ['error'],
});

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma;
}
```

1. Your DATABASE_URL_MAIN already has `?sslmode=require&channel_binding=require` — good. But ADD connection pooling params:

Plain Text

```
DATABASE_URL_MAIN=postgresql://...?sslmode=require&connection_limit=10&pool_tim
```

1. Create a unified health check endpoint that the admin dashboard uses:

TypeScript

```typescript
// api/admin/health.ts (or route.ts)
export async function GET() {
  try {
    await prisma.$queryRaw`SELECT 1`;
    return new Response(JSON.stringify({
      status: 'connected',
      timestamp: new Date().toISOString()
    }), { status: 200 });
  } catch (error: any) {
    return new Response(JSON.stringify({
      status: 'disconnected',
      error: error.message
    }), { status: 503 });
  }
}
```

1. In the admin dashboard component, use ONE consistent status check:

TypeScript

```typescript
useEffect(() => {
  const checkHealth = async () => {
    try {
      const res = await fetch('/api/admin/health');
      const data = await res.json();
      setDbStatus(data.status);  // 'connected' or 'disconnected'
    } catch {
      setDbStatus('disconnected');
    }
  };
  checkHealth();
  const interval = setInterval(checkHealth, 30000); // Re-check every 30s
  return () => clearInterval(interval);
}, []);
```

1. Remove ALL other scattered database status indicators that use different methods. Use ONLY this one health endpoint everywhere.

PROBLEM #10: ADMIN PANEL RESPONSIVENESS

THE FIX:

CSS

```css
/* Admin sidebar — collapsible on mobile */
.admin-sidebar {
  width: 260px;
  position: fixed;
  height: 100vh;
  left: 0;
  top: 0;
  z-index: 40;
  transform: translateX(-100%);
  transition: transform 0.3s ease;
}

.admin-sidebar.open {
  transform: translateX(0);
}

@media (min-width: 1024px) {
  .admin-sidebar {
    position: relative;
    transform: none;
  }
}

/* Admin tables — scroll horizontally on small screens */
.admin-table-container {
  width: 100%;
  overflow-x: auto;
  -webkit-overflow-scrolling: touch;
}

/* Admin stat cards — responsive grid */
.admin-stats {
  display: grid;
  grid-template-columns: 1fr;
  gap: 16px;
}
```

```
@media (min-width: 640px) {
  .admin-stats { grid-template-columns: repeat(2, 1fr); }
}

@media (min-width: 1024px) {
  .admin-stats { grid-template-columns: repeat(3, 1fr); }
}

@media (min-width: 1280px) {
  .admin-stats { grid-template-columns: repeat(4, 1fr); }
}
```

PROBLEM #11: PREMIUM UI POLISH (Make it look like a real brand)

THE FIX — Apply these design standards EVERYWHERE:

Plain Text

```
TYPOGRAPHY:
- Use consistent font: Inter or your brand font
- Headings: Bold, proper hierarchy (h1 > h2 > h3)
- Body text: 16px minimum on mobile

SPACING:
- Consistent padding: 16px mobile, 24px tablet, 32px desktop
- Section spacing: 64px between major sections
- Card padding: 16-24px

COLORS:
- Ensure text has 4.5:1 contrast ratio minimum
- Consistent brand color usage
- Hover/active states on ALL buttons and links

ANIMATIONS:
- Buttons: subtle scale on press (transform: scale(0.97))
- Cards: slight lift on hover (translateY(-2px) + shadow)
- Page transitions: fade in (opacity 0→1, 300ms)
- Loading states: skeleton screens, not blank white

SHADOWS:
- Cards: 0 2px 8px rgba(0,0,0,0.08)
- Modals: 0 20px 60px rgba(0,0,0,0.15)
- Buttons: 0 2px 4px rgba(0,0,0,0.1)
```

```
BORDER RADIUS:
- Buttons: 8px
- Cards: 12px
- Input fields: 8px
- Images: 8px
- Modals: 16px
```

PROBLEM #12: WATER BOTTLE 3D MISALIGNMENT

What's wrong: The `WaterBottleBody` in `garment3d.tsx` uses `CylinderGeometry` with UV mapping that doesn't align with the 2D SVG print zone. The `PhotoMockupMesh` (line 347) and the silhouette clip (DesignStudio.tsx line 2989) create drift.

THE FIX:

1. Ensure the UV coordinates on the CylinderGeometry map exactly to the `WATERBOTTLE` print zone in the 1000×1000 space
2. If using both 2D SVG preview and 3D preview, they MUST use the same coordinate system
3. The silhouette clip path must match the actual bottle outline precisely

---

FINAL VERIFICATION CHECKLIST — TEST ALL OF THESE:

Plain Text

```
[ ] Upload a design to T-shirt → it auto-fits centered in print zone
[ ] Tap/click the uploaded design → clean blue border + circle handles appear
[ ] Hold and drag the design → it moves smoothly (mouse AND touch)
[ ] Drag a corner handle → design resizes maintaining aspect ratio
[ ] Click outside the design → selection clears, handles disappear
[ ] Change T-shirt color from white to black → container stays SAME size
[ ] Switch from Front to Back view → print zone updates correctly
[ ] Design looks printed ON the product (blend mode working)
[ ] Print zone borders match actual product printable area
[ ] Repeat ALL above tests for Hoodie, Mug, Cap, Water Bottle
[ ] Homepage scrolls smoothly with mouse wheel on desktop
[ ] Homepage scrolls smoothly with touch on mobile
[ ] No horizontal overflow on ANY page at ANY screen size
[ ] Product grid is responsive (2 cols mobile, 3 tablet, 4 desktop)
[ ] Admin panel sidebar collapses on mobile
[ ] Admin dashboard shows "Connected" for database
[ ] All admin tables scroll horizontally on mobile
[ ] No console errors anywhere
```

```
[ ] Page loads in under 3 seconds
[ ] All buttons have hover/active states
[ ] All images load without layout shift
```

EXECUTION ORDER — DO THIS EXACTLY:

1. Fix the pointer event conflict in DesignStudio.tsx (Problem #1) — THIS IS THE #1 PRIORITY
2. Update print area coordinates in mockups.tsx (Problem #2)
3. Fix color change sizing (Problem #3)
4. Add auto-fit logic (Problem #4)
5. Add blend mode for realism (Problem #5)
6. Restyle selection handles (Problem #6)
7. Fix deselection on outside click (Problem #7)
8. Fix global responsiveness CSS (Problem #8)
9. Fix admin database status (Problem #9)
10. Fix admin responsiveness (Problem #10)
11. Apply premium UI polish (Problem #11)
12. Fix water bottle alignment (Problem #12)
13. Run through the ENTIRE verification checklist

DO NOT STOP UNTIL ALL 13 STEPS ARE COMPLETE AND ALL CHECKLIST ITEMS PASS.